Copy-Move Forgery Detection using a Siamese Swap-Test Quantum Neural Network

Phase-2 Progress & Evaluation Report

Final Year Project · Phase 2 | Date: 26 July 2026 | Pipeline: `cmfd_swaptest_pipeline.py` (v2)

1. Executive Summary

This report documents the current state of a quantum-machine-learning pipeline for **copy-move forgery detection (CMFD)** — detecting regions of an image that have been duplicated and pasted elsewhere in the same image. The system pairs a classical patch descriptor with a **Siamese swap-test quantum neural network (QNN)** that learns a similarity score between two image patches, followed by a spatial offset-consistency filter that keeps only genuine duplications.

> **Headline finding.** The infrastructure (data loading, patch extraction, QNN similarity, metrics, multi-dataset evaluation) is **sound and runs end-to-end**. However, the model currently **over-detects severely** — it flags 50–97% of each test image as forged. Consequently the previously-remembered "~80%" figure was **recall, not accuracy**; true pixel accuracy on those samples is ~3–12%, and localization quality (F1/IoU) is low. The remaining work is **detection quality**, not plumbing.

2. Objective & Target Output

The intended end product: *given any input image, decide whether it is forged, and if so, localize the exact duplicated region as a bounding box or heatmap.* That decomposes into four capabilities, tracked in the gap analysis (§8).

3. System Architecture

The architecture is a **pairwise similarity model**, which is the structurally correct choice for copy-move (the task is inherently "does patch A match patch B elsewhere?"). A single-patch classifier cannot express duplication. An earlier degenerate embedding (fixed amplitude embedding + data-independent tied unitary, which the swap test provably cancels) was replaced with a **data re-uploading** ansatz so the tied weights have a genuine effect on the measured overlap.

Component	Specification
Quantum registers	**4 qubits per register**, two registers (patch A, patch B) = 8 qubits **+ 1 ancilla** for the swap test = **9 wires total**
Trainable parameters	**24** — shape (2 layers × 4 qubits × 3 rotations RX/RY/RZ); the same weight tensor is **tied** across both registers (Siamese)
Similarity readout	Swap test: ancilla $\langle Z \rangle = \lvert\langle \psi_A \rvert \psi_B \rangle\rvert^2$, a learned similarity in [0,1]
Patches	**32×32** px, stride **8**, sliding window → **841 patches** on a 256×256 image
Patch descriptor	**24-dim**: 12 largest-magnitude DCT coefficients + 12-bin intensity histogram, L2-normalized, angle-encoded via data re-uploading
Candidate pairing	O(N log N) lexicographic block-matching (Fridrich-style) — sort by feature, compare within a sliding window, instead of $O(N^2)$ all-pairs
False-match suppression	Offset-consistency vote over (dy, dx): genuine copy-move produces many pairs sharing one offset (translation only — rotation is a documented TODO)
Binary mask	GT: grayscale → NEAREST resize → threshold >128. Prediction: fill matched-pair patches → morphological close (dilate→erode)
Training	Self-supervised synthetic copy-move pairs (parameter-shift gradients, MSE on similarity labels)

Is the architecture correct? Conceptually yes for *measuring patch similarity*, and the degeneracy bug is fixed. But it is **incomplete as a detector**: (i) there is no image-level forged/authentic decision, (ii) the DCT+histogram descriptor is not rotation/scale invariant, and (iii) it is not yet shown that the quantum similarity outperforms a classical distance on the same features (an ablation is needed).

4. Datasets & Their Roles

Training and evaluation must use **different** data sources — a recurring point of confusion. The Siamese similarity *must* be trained on synthetic pairs because real CMFD datasets provide no patch-level correspondence ground truth.

Dataset	Role	Why
Synthetic (self-supervised)	**Training — mandatory**	Only source of labeled positive/negative patch pairs; CoMoFoD masks have no correspondence labels
CoMoFoD (small v2)	**Primary evaluation (headline)**	Real forgeries + real binary masks; loader already matches it (5,000 forged images)
COVERAGE (100 pairs)	**Stress test only**	Adversarial by design (similar genuine objects); useful to prove generalization, but not a headline metric

Recommendation: train on synthetic, report on **CoMoFoD**. Never evaluate on synthetic data (it only proves the model learned its own augmentation). Keep COVERAGE as a "hard case" data point.

5. Evaluation Methodology — why not accuracy?

Forged regions occupy only **~2–11% of pixels**, so the pixel classes are extremely imbalanced. Under this imbalance, plain accuracy is **actively misleading**: a model that predicts "nothing is forged" scores ~90–98% accuracy while detecting **zero** forgeries.

Metric	Definition	Meaning here
Accuracy	(TP+TN) / all	**Misleading — rewards predicting nothing**
Recall	TP / (TP+FN)	Fraction of true forged pixels *found*. **Gamed by flagging everything** → this is the "80%" trap
Precision	TP / (TP+FP)	Of the pixels flagged, how many were actually forged
F1	2·P·R / (P+R)	**Honest single scalar — collapses if precision OR recall is bad**
IoU (Jaccard)	TP / (TP+FP+FN)	**Localization overlap — directly measures "does the predicted region match the true region?"**

> **Reporting rule for this project:** lead with **F1** and **IoU**; cite precision+recall as the pair that explains F1; never headline accuracy. IoU is literally the metric that measures the stated goal (exact forged area).

6. Results

6.1 COVERAGE — loose parameters (threshold 0.8, votes 3, 384 px)

Image	Accuracy	Precision	Recall	F1	IoU
COVERAGE 1t	0.277	0.269	1.000	0.424	0.269
COVERAGE 10t	0.092	0.055	1.000	0.105	0.055
COVERAGE 100t	0.344	0.337	1.000	0.505	0.337
Mean	**0.238**	**0.220**	**1.000**	**0.345**	**0.221**

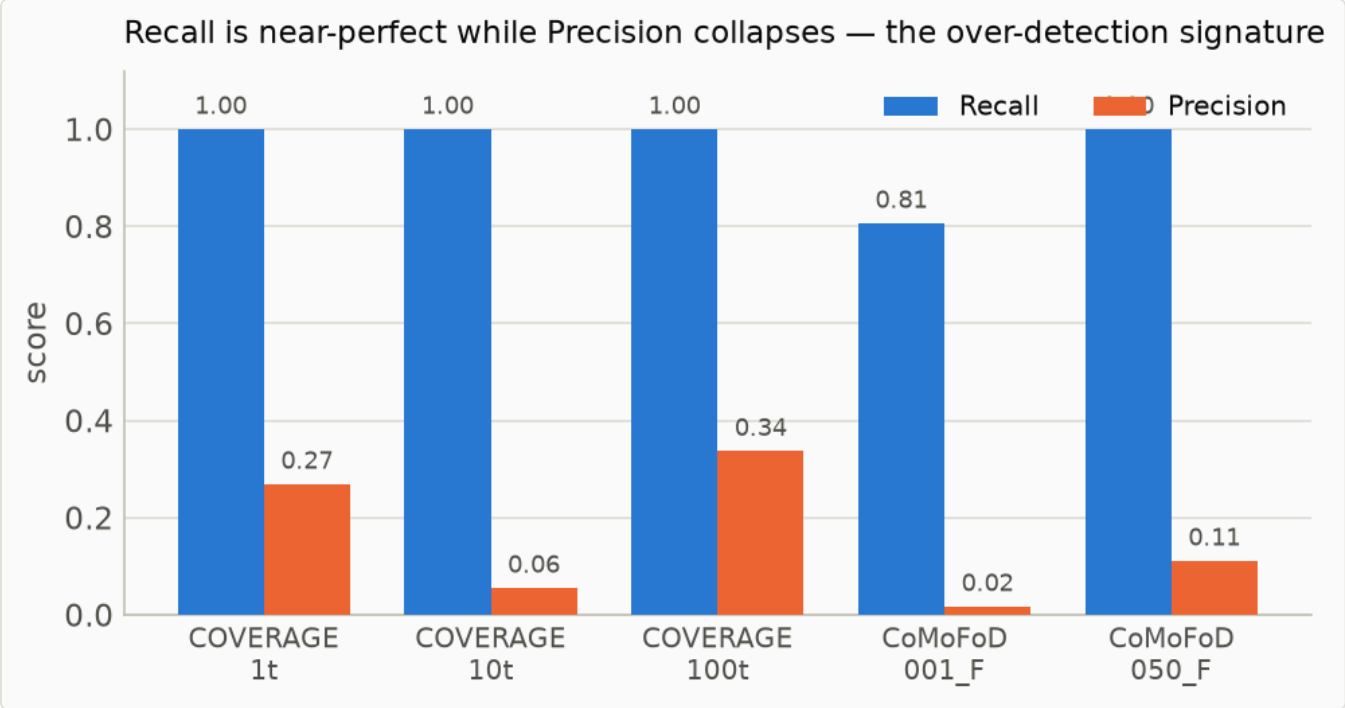


Figure 1. Across every test image, recall is ~1.0 while precision is far lower — the model finds the forgery only because it flags almost the whole image. (COVERAGE at loose parameters; CoMoFoD from saved masks, §6.3.)

6.2 Effect of tightening parameters (COVERAGE, threshold 0.92, votes 8, 256 px)

Image	Precision	Recall	F1	IoU
COVERAGE 1t	0.308	0.994	0.471	0.308
COVERAGE 10t	0.102	1.000	0.185	0.102
COVERAGE 100t	0.259	0.420	0.320	0.191
Mean	**0.223**	**0.805**	**0.325**	**0.200**

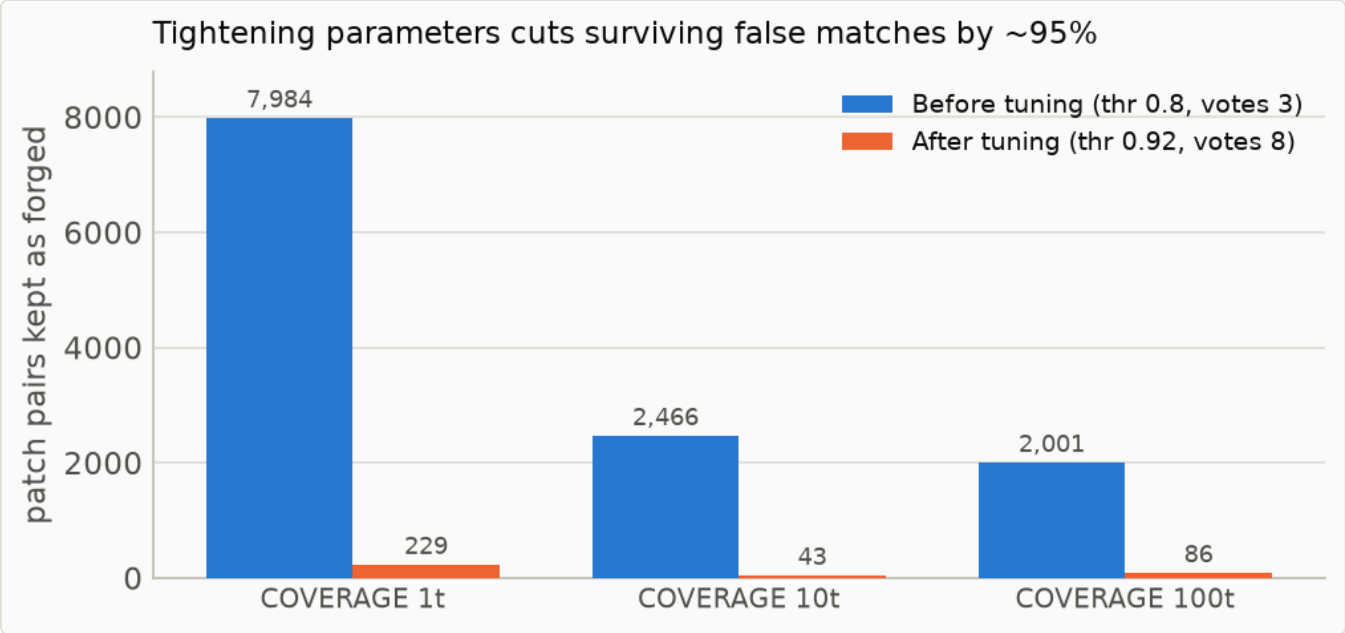


Figure 2. The mechanism improves sharply: the offset-consistency filter now removes ~95% of candidate matches (before → after: 7,984→229, 2,466→43, 2,001→86). Mean F1 looks flat only because image 100t swung from "flag everything" (recall 1.0) to under-detecting (recall 0.42).

6.3 CoMoFoD — recomputed from saved prediction masks

No CoMoFoD metrics were previously persisted; the two masks below were recovered from an earlier run (loose parameters) and re-scored against ground truth.

Image	Accuracy	Precision	Recall	F1	IoU	Forged area
001_F	0.030	0.017	0.806	0.032	0.017	2.0%
050_F_BC1	0.125	0.111	1.000	0.200	0.111	10.9%

This resolves the "80%" question. Image 001_F scored **recall 0.806** — the figure in memory — but with **precision 0.017** and **accuracy 0.030**, because ~97% of the image was flagged (63,346 false-positive pixels out of 65,536). CoMoFoD floods exactly like COVERAGE, so the over-detection is a **model/threshold problem, not a dataset-specific one**.

7. Key Finding — systemic over-detection

The predicted mask covers most of each image. The diagnostic signature is consistent: **recall ≈ 1.0, precision near the forged-area base rate**. The offset-consistency filter, which is supposed to suppress coincidental matches, barely rejects anything at loose thresholds (e.g. 9,734→7,984 pairs). On self-similar texture, false matches cluster into offset bins just like true ones, so they survive.

Two compounding causes: (1) the DCT+histogram descriptor is weak — it cannot separate a true duplicate from a genuinely similar region; (2) thresholds/resolution were not tuned for the test conditions. Tightening parameters (§6.2) confirms the mechanism is fixable — surviving false matches dropped ~95% — but precision remains low, pointing at the descriptor as the deeper bottleneck.

8. Gap Analysis — distance to the final goal

Capability	Status	Reality
1. Input any image + preprocess	DONE	Loaders + patch extraction work on any image/dataset
2. Image-level "forged: yes/no"	NOT BUILT	Detector always returns a mask; nothing decides "authentic". No classifier head
3. Localization output format (box / heatmap)	HALF	Produces a binary mask; converting to a box/heatmap is trivial (~10 lines)
4. Localization accuracy (exact edge/area)	NOT MET	Floods 50–97% of the image (precision 0.05–0.34) — the core unsolved problem

> **Overall ≈ 35% of a usable output.** Scaffolding is ~60–70% built, but the parts that make it *a detector* — discriminating forged-vs-clean and localizing precisely — are the hard remaining work. The gap is detection *quality*, not output formatting.

9. Recommendations

- **Datasets:** keep training on **synthetic** pairs (mandatory); report headline metrics on **CoMoFoD**; use **COVERAGE** only as a stress test.
- **COVERAGE — keep it.** ~142 MB extracted, the loader reads it, and a second dataset strengthens the report. Optionally delete only the 142 MB download zip (`OneDrive_1_7-26-2026.zip`) to reclaim space.
- **Metrics:** report F1 + IoU (with precision/recall); drop accuracy from headline claims.

10. Roadmap — next steps

1. **Establish an honest baseline** — run a 20–50 image CoMoFoD batch with tuned parameters and record mean F1/IoU. (Nothing trustworthy exists yet.)

2. **De-flood localization** — tune `sim-threshold` / `min-offset-votes`; strengthen the offset filter (e.g. RANSAC on offsets); if precision still stalls, replace/augment the descriptor with a rotation/scale-invariant feature (e.g. Zernike moments).
3. **Add the image-level decision** — derive forged/authentic from the offset-consistent matched area, calibrated on authentic images to bound false positives.
4. **Ablation** — confirm the swap-test QNN beats a classical similarity on the same features; otherwise justify the quantum component explicitly.
5. **Final output format** — convert the mask to a bounding box (connected components) or a similarity heatmap. Trivial, and correctly done last.

> **Single most important next action:** steps 1–2 — get a real CoMoFoD number, then kill the over-detection. Everything else builds on that.

Appendix A — exact run configurations

```
COVERAGE, loose (§6.1):
  --forged-glob '*t.tif' --mask-union
  --sim-threshold 0.8  --min-offset-votes 3  --target-long-side 384

COVERAGE, tuned (§6.2):
  --forged-glob '*t.tif' --mask-union
  --sim-threshold 0.92 --min-offset-votes 8  --target-long-side 256

Model checkpoint: checkpoints_full/swaptest_qnn_epoch10.npy
  weights shape (2, 4, 3) = 24 params  |  9 wires (4+4 registers + 1 ancilla)
Patch config: 32x32, stride 8  ->  841 patches @ 256x256
```

Appendix B — file inventory

Path	Purpose
`code/cmfd_swaptest_pipeline.py`	Core pipeline: QNN, detector, metrics, training
`code/grip_dataset.py`	Dataset-agnostic loader (auto-discovery, mask-union, aspect-preserving resize)
`code/test_grip.py`	Evaluation runner (native-res + tuning flags)
`data/comofod/`	CoMoFoD small v2 (primary evaluation)
`data/coverage/`	COVERAGE (100 pairs, stress test)
`results_coverage/`, `results_coverage_tuned/`	COVERAGE run outputs (masks + CSV)